

Name of the Student	G.NAGA MANIKANTA
Reg. No	192425310
GitHub Link	

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

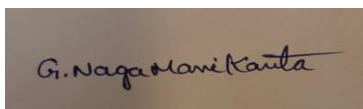
Course Outcome Weightage: 10%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I _____G.NAGA MANIKANTA(192425310)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student:MANIKANTA

Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100%
Q1						
Q2						

Q3						
Q4						
Q5						
Total						

QUESTIONS:

05

Total Marks:25

1. Desing and develop a model for a hospital has only **2,000 labeled X-ray images** for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. **Design a CNN-based transfer learning model** by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

ANSWER:

A hospital has only 2,000 labeled X-ray images for classifying images into **normal** and **abnormal** categories. Since the dataset is small, training a CNN from scratch may lead to overfitting. Therefore, a **pre-trained ResNet** can be used through transfer learning.

Model Design

A pre-trained ResNet model is first loaded with weights learned from a large dataset such as ImageNet.

Layers to freeze:

- Freeze most of the early and middle ResNet convolutional layers.
- These layers learn general features such as edges, textures, shapes, and patterns that can also be useful for X-ray images.
- Initially, the weights of these layers are not updated during training.

Layer to replace:

- Remove the original ResNet classification layer.
- Replace it with a new classification head suitable for two classes.

A possible architecture is:

X-ray Image → ResNet Frozen Layers → Global Average Pooling → Fully Connected Layer → Dropout → 2-Class Output

The final layer can contain **2 neurons with Softmax activation**, producing probabilities for:

- Normal
- Abnormal

Training Process

1. Resize all X-ray images to the input size required by ResNet.
2. Normalize the images according to the pre-trained model requirements.
3. Apply data augmentation such as rotation, scaling, cropping, and horizontal flipping where medically appropriate.
4. Freeze the ResNet feature-extraction layers.
5. Train only the newly added classification layers.
6. Use **cross-entropy loss** and an optimizer such as Adam.
7. Use validation data and early stopping to prevent overfitting.
8. After the new classification head has learned reasonable features, some of the deeper ResNet layers can be **unfrozen** and fine-tuned using a very small learning rate.

Conclusion

Transfer learning is suitable because only 2,000 labeled images are available. The pre-trained ResNet provides useful feature representations, while fine-tuning the deeper layers allows the model to adapt to X-ray-specific patterns. This reduces training time and generally provides better generalization than training a CNN completely from scratch.

2. An agricultural organization wants to identify **five types of plant diseases** from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. **Design a DenseNet-based classification model** and explain how dense connections can help the model reuse features during classification.

ANSWER :

An agricultural organization needs to classify leaf images into **five plant disease categories**. Since the number of training images is limited, a DenseNet model is suitable because its dense connections promote efficient feature reuse.

Model Design

A pre-trained DenseNet can be used as the feature extractor.

Architecture:

Leaf Image → Convolution → Dense Blocks → Transition Layers → Global Average Pooling → Fully Connected Layer → 5-Class Softmax

The final classification layer of the pre-trained DenseNet is replaced with a new fully connected layer containing **5 output neurons**, one for each disease class.

Dense Connections

The main feature of DenseNet is that each layer inside a dense block receives the outputs of **all preceding layers** as input.

For example:

Layer 1 → Layer 2 → Layer 3 → Layer 4

In a conventional network, Layer 4 mainly receives information from Layer 3. In DenseNet, Layer 4 receives information from:

Layer 1 + Layer 2 + Layer 3

Thus, the network can directly reuse previously learned features.

How Feature Reuse Helps

Early layers may detect simple features such as:

- edges
- colors
- leaf boundaries
- textures

Later layers can combine these features to recognize:

- spots
- discoloration
- lesions
- disease patterns

Dense connections allow these earlier features to be reused instead of learning them repeatedly. This reduces redundant learning and improves information and gradient flow through the network.

Training Process

1. Resize and normalize leaf images.
2. Apply data augmentation such as rotation, flipping, cropping, and brightness changes.
3. Load a pre-trained DenseNet.
4. Freeze most of the initial layers.
5. Replace the original classification layer with a 5-class classifier.
6. Train the new classification layer.
7. Optionally fine-tune deeper DenseNet layers using a small learning rate.
8. Use categorical cross-entropy loss and an optimizer such as Adam.

Conclusion

DenseNet is effective for limited datasets because its dense connections encourage **feature reuse, efficient parameter utilization, and improved gradient propagation**. This makes it suitable for recognizing different disease patterns in plant leaves.

3. A university wants to predict a student's future academic performance using their **weekly attendance, assignment marks, internal assessment marks, and previous examination scores** collected over several weeks. **Design an LSTM-based model** that can process this sequential information and predict the student's performance category.

ANSWER:

The university has data collected over several weeks, including **attendance, assignment marks, internal assessment marks, and previous examination scores**. Since the data has a time-dependent sequence, an **LSTM (Long Short-Term Memory)** network can be used.

Input Representation

For every week, create a feature vector:

$X_t = [\text{Attendance, Assignment Marks, Internal Marks, Previous Exam Score}]$

For example:

Week 1 → Week 2 → Week 3 → ... → Week N

These weekly feature vectors form a sequence given to the LSTM.

Model Architecture

Weekly Student Data → LSTM Layer → Dropout → Fully Connected Layer → Softmax Output

For example:

- Input shape: (number of weeks, 4 features)
- LSTM layer: 64 or 128 hidden units
- Dropout layer: for reducing overfitting
- Dense layer
- Softmax output layer

If the categories are:

Poor / Average / Good / Excellent

the final layer will have **4 neurons**.

Working of LSTM

At each time step, the LSTM receives the current week's data and maintains information from previous weeks.

The LSTM uses three major gates:

- **Forget gate:** decides which old information should be discarded.
- **Input gate:** decides which new information should be stored.
- **Output gate:** determines what information should be passed to the next stage.

For example, if a student's attendance has been consistently decreasing for several weeks, the LSTM can learn this temporal pattern and use it when predicting future performance.

Training Process

1. Collect weekly student records.
2. Normalize numerical features.
3. Arrange each student's records chronologically.

4. Create training, validation, and test sequences.
5. Feed the sequences into the LSTM.
6. Use categorical cross-entropy loss for multiple performance categories.
7. Train using Adam or another suitable optimizer.
8. Evaluate the model using accuracy, precision, recall, and F1-score.

Conclusion

LSTM is appropriate because student performance depends not only on the current week's data but also on **patterns observed over previous weeks**. Its memory mechanism enables the model to learn these temporal dependencies and predict the student's future performance category.

4. An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, **design a Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.

ANSWER:

An organization wants to classify customer reviews into **positive, negative, or neutral**. The meaning of a word can depend on both the words before and after it. Therefore, a **Bidirectional RNN (BiRNN)** is suitable.

Major Processing Stages

Review Text → Tokenization → Word Embedding → Bidirectional RNN → Dense Layer → Softmax → Sentiment

1. Text Preprocessing

The customer review is first cleaned and divided into tokens.

For example:

"The product is not very good"

can be converted into individual tokens:

The | product | is | not | very | good

These tokens are converted into numerical representations.

2. Word Embedding

Each word is converted into a dense numerical vector using an embedding layer.

Thus:

Words → Word Vectors

The embedding allows the model to represent semantic relationships between words.

3. Bidirectional RNN

The BiRNN contains two RNNs:

Forward RNN: processes the sentence from left to right.

Backward RNN: processes the sentence from right to left.

Their hidden representations are combined.

Therefore, when processing a word, the model can use information from both its **past and future context**.

For example:

"The movie was not good"

The word **"good"** is interpreted together with the preceding word **"not"**, helping the model recognize the negative sentiment.

4. Dense Layer

The combined representation from the BiRNN is passed to a fully connected layer.

5. Output Layer

A Softmax layer produces probabilities for:

- Positive
- Negative
- Neutral

The class with the highest probability becomes the final sentiment prediction.

Training

The model is trained using labeled customer reviews and **categorical cross-entropy loss**. Techniques such as dropout can be used to reduce overfitting.

Conclusion

A BiRNN is effective for sentiment analysis because it considers contextual information from **both directions**, allowing it to better understand the meaning of words in a complete sentence.

5. A software company wants to develop a system that translates **English sentences into Hindi**. The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. **Design an Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

ANSWER:

A software company wants to translate English sentences into Hindi. Since the input and output sentences can have different lengths, an **Encoder–Decoder architecture** is appropriate.

The model can be implemented using an **RNN, LSTM, or GRU** encoder and decoder.

Overall Architecture

English Sentence → Encoder → Hidden Representation → Decoder → Hindi Sentence

For example:

English: "I am going to college."

↓

Encoder

↓

Hidden Representation

↓

Decoder

↓

Hindi: "मैं कॉलेज जा रहा हूँ।"

1. Encoder

The encoder receives the English sentence one word at a time.

For example:

I → am → going → to → college

At every time step, the encoder updates its hidden state.

The encoder's purpose is to convert the complete input sentence into a meaningful internal representation.

2. Hidden Representation

The final hidden state or context representation contains information about the input sentence.

It represents important information such as:

- words in the sentence
- their relationships
- sentence context
- overall meaning

This representation is passed to the decoder.

3. Decoder

The decoder generates the Hindi translation **one word at a time**.

For example:

<START> → मैं → कॉलेज → जा → रहा → हूँ → <END>

At every step, the decoder uses:

- its previous hidden state
- the previous output word
- the encoded representation

to predict the next Hindi word.

4. Output Layer

The decoder's output is passed through a fully connected layer followed by **Softmax**.

The Softmax layer produces probabilities for all Hindi words in the vocabulary.

For example:

मैं: 0.70

हम: 0.10

वह: 0.05

...

The word with the highest probability is selected as the next output.

Handling Different Sentence Lengths

The Encoder–Decoder architecture does not require the English and Hindi sentences to have the same number of words.

For example:

English: "How are you?"

Hindi: "आप कैसे हैं?"

The number and order of words can differ because the encoder first represents the input sequence and the decoder independently generates the output sequence.

Training

The model is trained using a large collection of paired English-Hindi sentences.

During training:

1. The English sentence is given to the encoder.
2. The encoder produces the hidden representation.
3. The decoder receives the representation and generates Hindi words.
4. The predicted words are compared with the actual Hindi sentence.
5. The error is calculated using cross-entropy loss.
6. Backpropagation through time updates the model parameters.

For better performance on long sentences, an **attention mechanism** can be added so that the decoder can focus on relevant parts of the input sentence while generating each output word.

Conclusion

The Encoder–Decoder model is well suited for machine translation because it separates the process into two stages: the **encoder understands the input sentence**, while the **decoder generates the translated sentence**. Since the decoder generates its output sequentially, the input and output sentences can have different lengths.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
		solve the problem			
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis